

Response Items: The Building Blocks

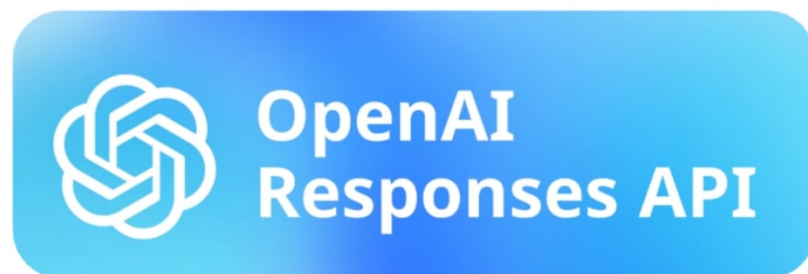
WORKING WITH THE OPENAI RESPONSES API



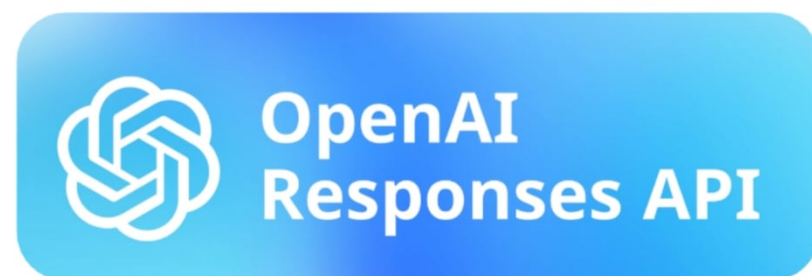
James Chapman

AI Curriculum Manager, DataCamp

The Journey

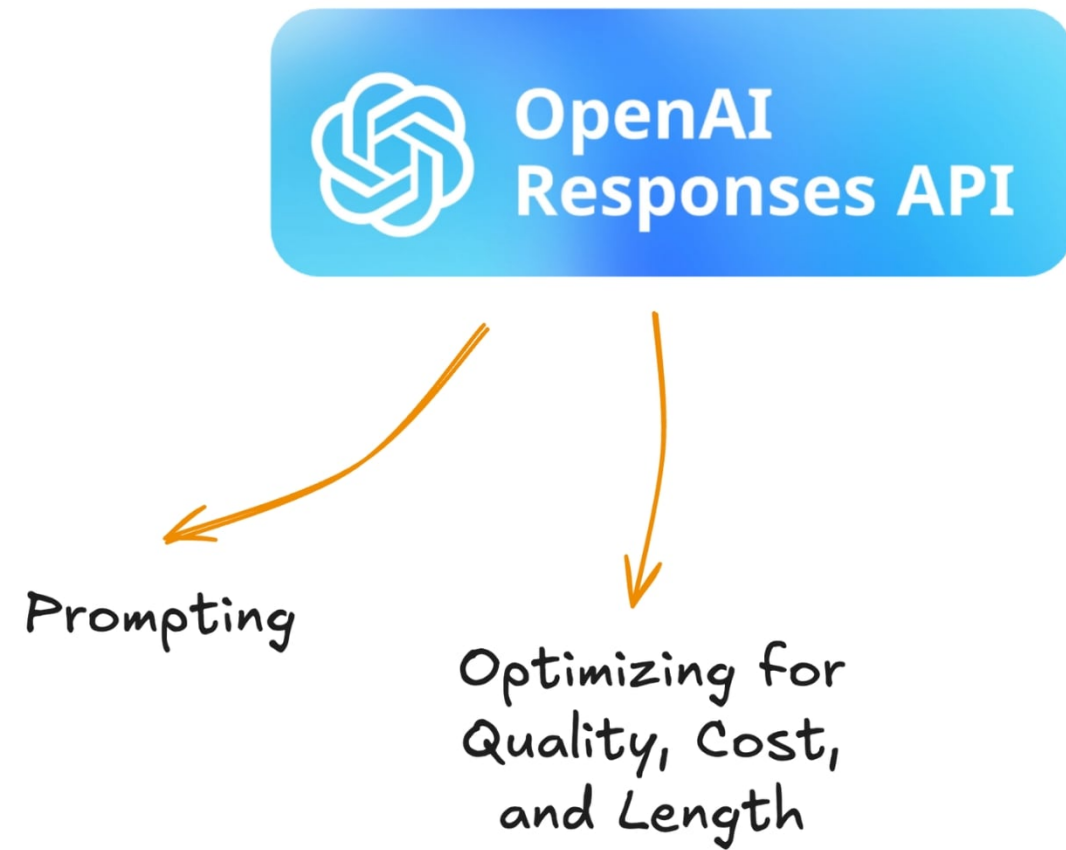


The Journey

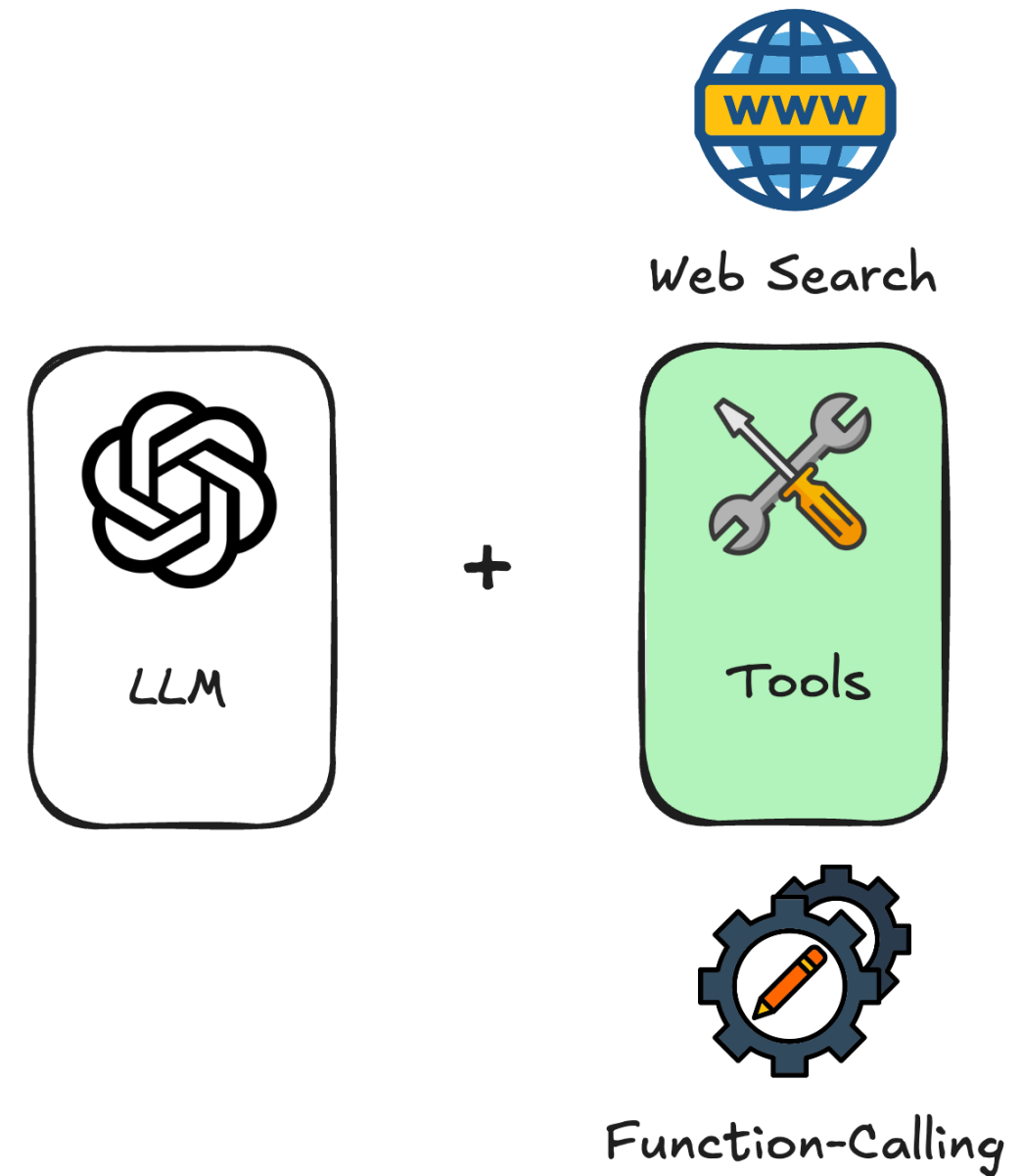
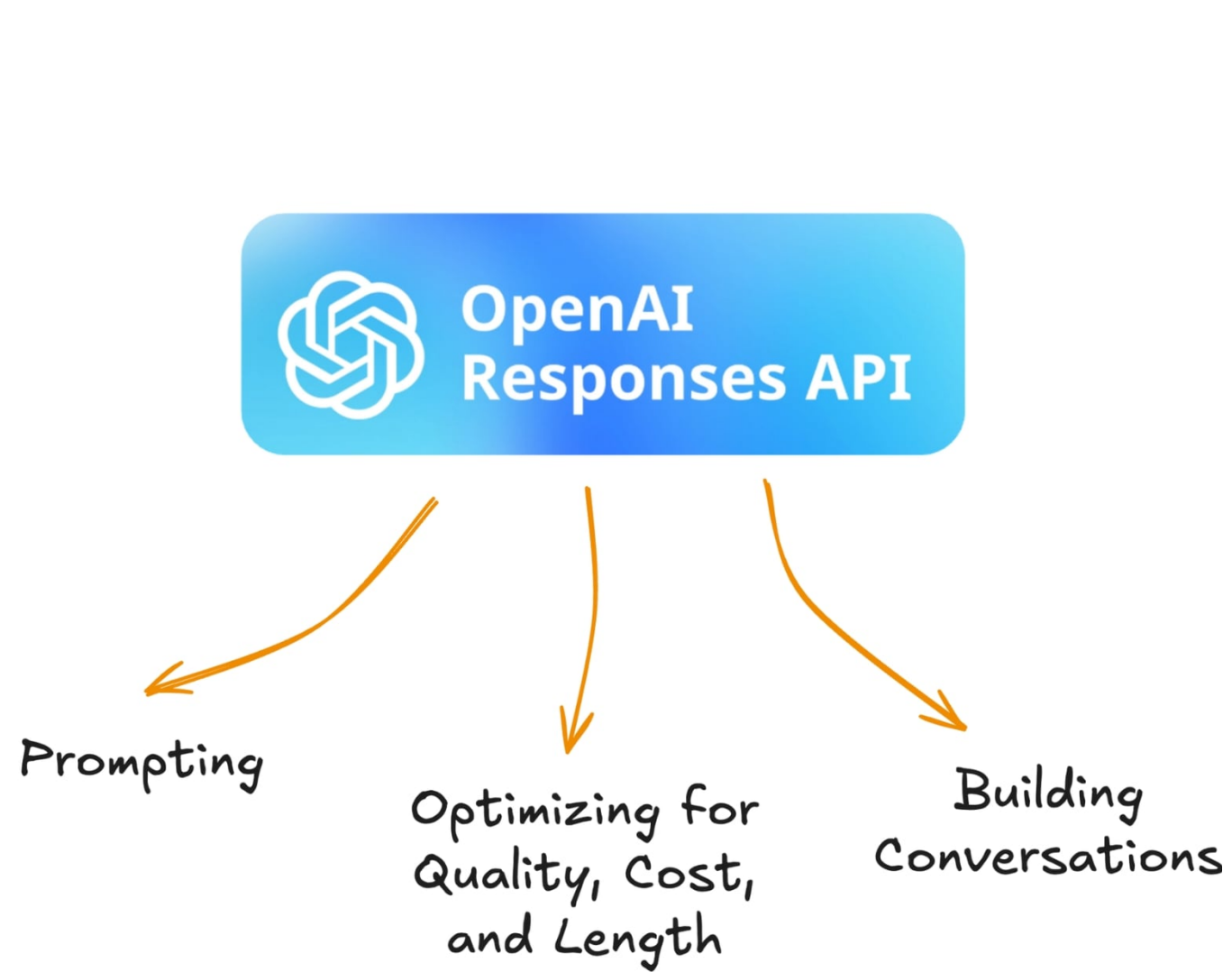


Prompting

The Journey



The Journey



Your Item Toolbox

```
for item in response.output:  
    print(item)
```

```
ResponseReasoningItem(id='rs_0471305df20125be00692440df2ed88193bd532a15ee6ab7dc',  
                      summary=[], type='reasoning', content=None, encrypted_content=None,  
                      status=None)
```

```
ResponseOutputMessage(id='msg_0471305df20125be00692440df642481938921f6bd2c119688',  
                      content=[ResponseOutputText(annotations=[], text='The OpenAI...',  
                                                    type='output_text', logprobs=[])],  
                      role='assistant', status='incomplete', type='message')
```

Items as Building Blocks

```
for item in response.output:  
    if item.type == "message":  
        if item.status == "incomplete":  
            print("Message cut-off!")
```

Message cut-off!

Items as Building Blocks

```
for item in response.outputs:
    if item.type == "reasoning":
        if item.summary:
            print(f"Reasoning Summary: {item.summary[0]}")
        else:
            print("No reasoning summary available.")
    if item.type == "message":
        print(f"Assistant: {item.content[0].output_text}")
```

```
No reasoning summary available.
Assistant: The OpenAI...
```

Coming up...

Current Item Types

- 'reasoning'
- 'message'

```
for item in response.output:  
    if item.type == "message":  
        if item.status == "incomplete":  
            print("Message cut-off!")
```

New Item Types

- 'web_search_call'
- 'function_call'
- 'function_call_output'

Let's practice!

WORKING WITH THE OPENAI RESPONSES API

Web Search That Just Works

WORKING WITH THE OPENAI RESPONSES API

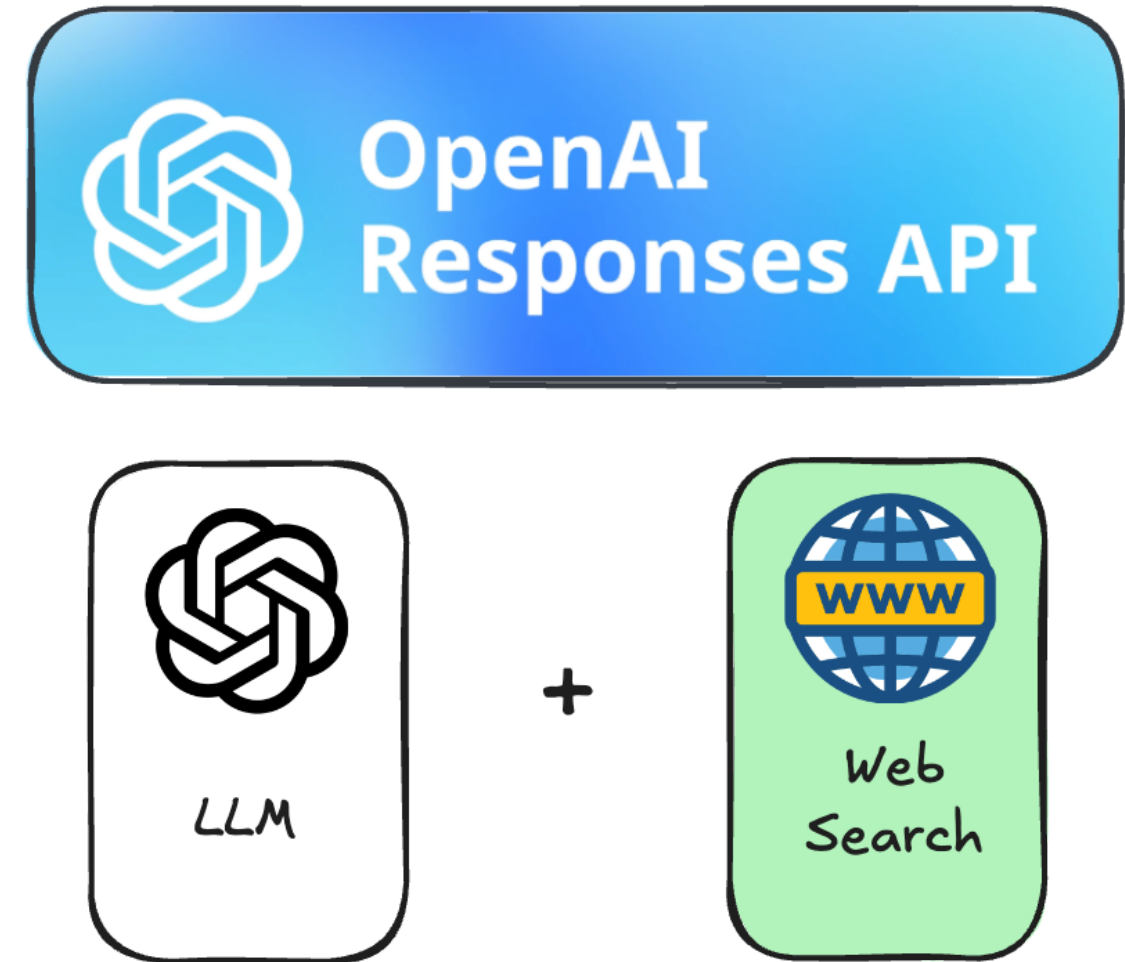


James Chapman

AI Curriculum Manager, DataCamp

The Need for Up-to-Date Information

- LLMs → have a *knowledge cut-off*
 - Can be >1yr old:
 - e.g., GPT 4.1 is June, 2024
 - Problematic for coding, research, and more...



Knowledge Cutoffs in Practice

```
response = client.responses.create(  
    model="gpt-5.4-mini",  
    input="What is the current temperature in Tokyo, Japan?"  
)  
  
print(response.output_text)
```

```
I can't access real-time data, so I can't tell you the current temperature in  
Tokyo right now.
```

Getting Started with Web Search

```
response = client.responses.create(  
    model="gpt-5.4-mini",  
    tools=[{"type": "web_search"}],  
    input="What is the current temperature in Tokyo, Japan?"  
)  
  
print(response.output_text)
```

```
Current temperature in Tokyo (Japan): 46°F (8°C), foggy. (As of Nov 25, 2025,  
Tokyo time.)
```

Web Search Under-the-Hood

```
for item in response.output:  
    print(item)
```

```
ResponseReasoningItem(id='...', summary=[], type='reasoning', content=None,  
                      encrypted_content=None, status=None)  
ResponseFunctionWebSearch(id='...', action=ActionSearch(query='weather: Tokyo, Japan',  
                                                         type='search', sources=None), status='completed',  
                          type='web_search_call')  
ResponseReasoningItem(id='...', summary=[], type='reasoning', content=None,  
                      encrypted_content=None, status=None)  
ResponseOutputMessage(id='...', content=[ResponseOutputText(annotations=[],  
                                                             text='Current temperature in Tokyo (Japan)...', type='output_text',  
                                                             logprobs=[])], role='assistant', status='completed', type='message')
```

Including All Sources

```
response = client.responses.create(
    model="gpt-5.4-mini",
    tools=[{"type": "web_search"}],
    input="What is the current temperature in Tokyo, Japan?",
    include=["web_search_call.action.sources"]
)
for item in response.output:
    if item.type == "web_search_call":
        print(item)
```

```
ResponseFunctionWebSearch(id='...', action=ActionSearch(query='weather: Tokyo, Japan',
    type='search', sources=[ActionSearchSource(type='api', url=None,
    name='oai-weather')])), status='completed', type='web_search_call')
```

Other Source Types

```
response = client.responses.create(  
    model="gpt-5.4-mini",  
    tools=[{"type": "web_search"}],  
    input="What is the latest Python version released?",  
    include=["web_search_call.action.sources"]  
)  
  
for item in response.output:  
    if item.type == "web_search_call":  
        print(item.action.sources)
```

Other Source Types

```
[{'type': 'url', 'url': 'https://www.python.org/doc/versions/'},  
 {'type': 'url', 'url': 'https://test.python.org/doc/versions/'},  
 {'type': 'url', 'url': 'https://www.techradar.com/pro/...'}]  
  
[{'type': 'url', 'url': 'https://www.python.org/downloads/latest/python3.14/'},  
 {'type': 'url', 'url': 'https://www.python.org/downloads/release/python-3140rc1/'},  
 {'type': 'url', 'url': 'https://blog.python.org/2025/10/python-3140-final-is-here.html'},  
 {'type': 'url', 'url': 'https://www.python.org/downloads/release/python-3140rc2/'},  
 {'type': 'url', 'url': 'https://peps.python.org/pep-0745/'},  
 {'type': 'url', 'url': 'https://blog.python.org/2025/05/python-3140-beta-1-is-here.html'},  
 {'type': 'url', 'url': 'https://www.python.org/downloads/release/python-3140rc3/'},  
 {'type': 'url', 'url': 'https://blog.python.org/2025/10/'},  
 {'type': 'url', 'url': 'https://blog.python.org/2025/07/python-314-release-candidate-1-is-go.html'},  
 {'type': 'url', 'url': 'https://blog.python.org/2025/01/python-3140-alpha-4-is-out.html'},  
 {'type': 'url', 'url': 'https://mail.python.org/archives/list/python-announce-list...'}]
```

Let's practice!

WORKING WITH THE OPENAI RESPONSES API

Function-Calling Tools: Defining the Function

WORKING WITH THE OPENAI RESPONSES API

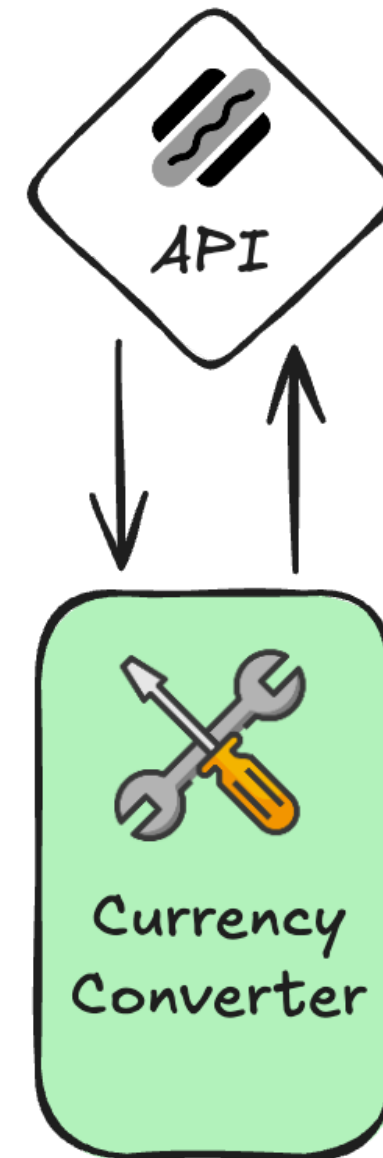


James Chapman

AI Curriculum Manager, DataCamp

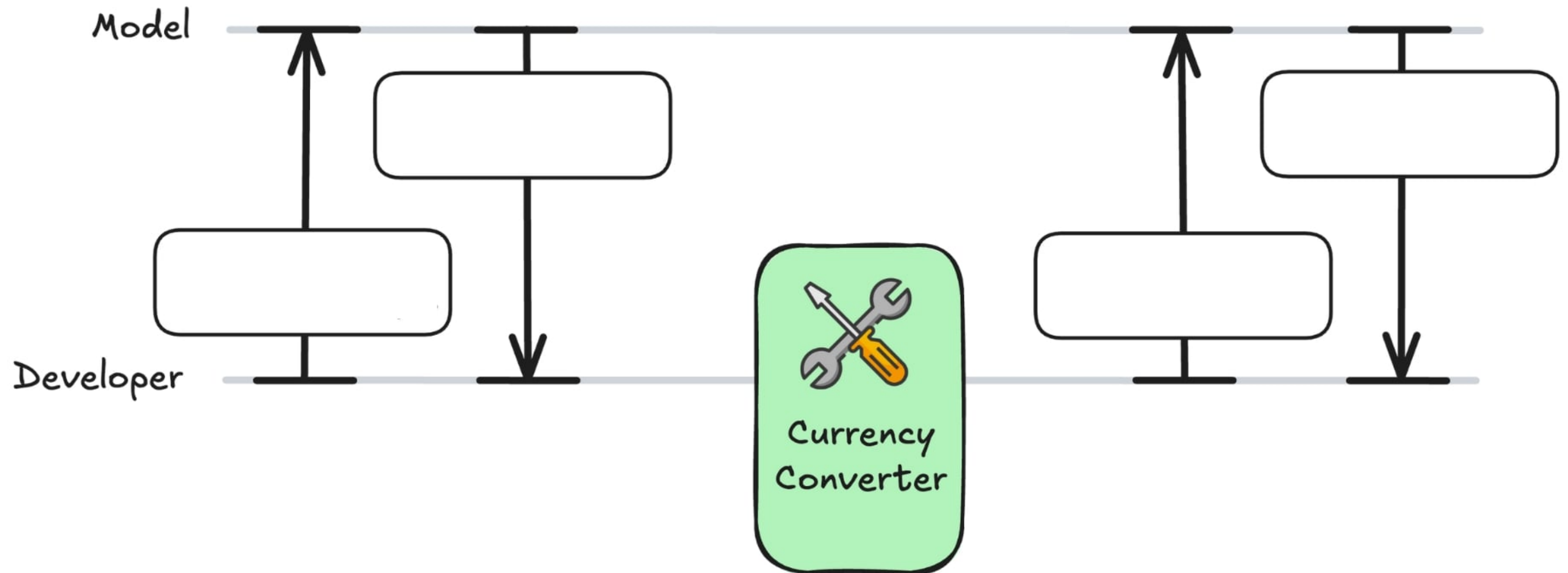
Function-Calling Tools

- **Function-Calling Tools:** LLMs calling programmed functions
 - Can be created for almost anything!
- `convert_currency()`
 - `amount` : amount of money
 - `from_currency` : original currency
 - `to_currency` : target currency

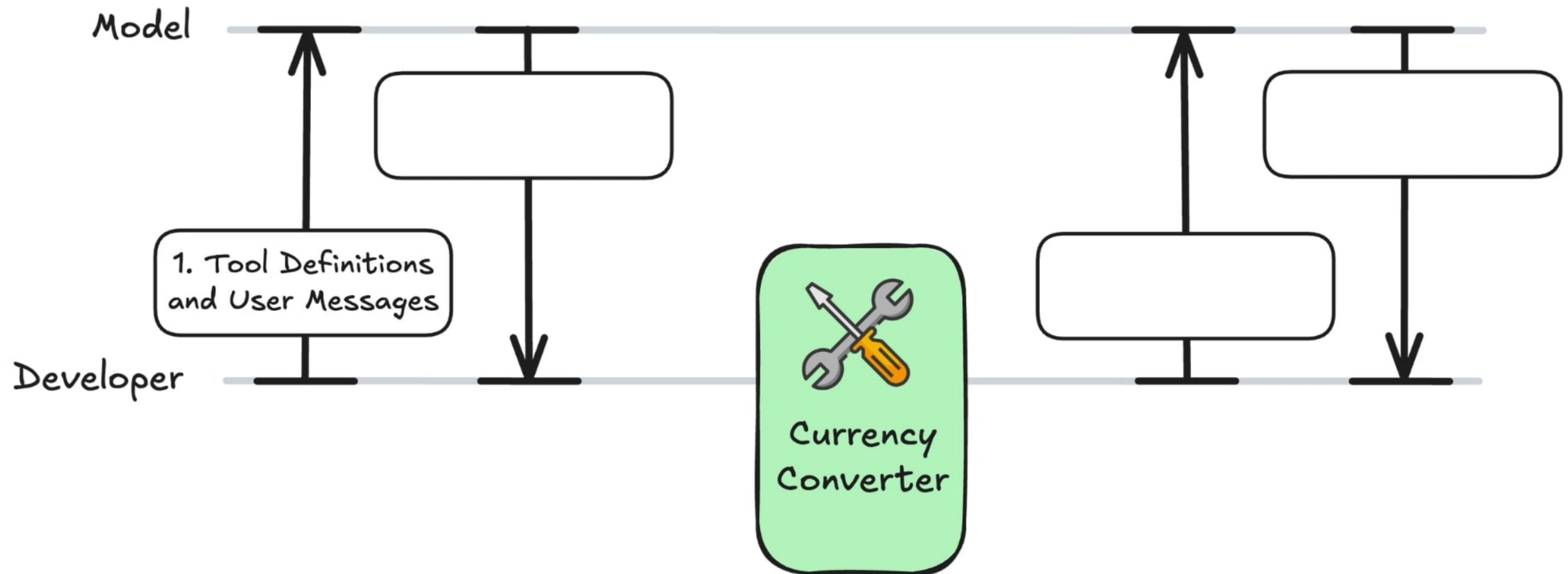


¹ <https://frankfurter.dev/>

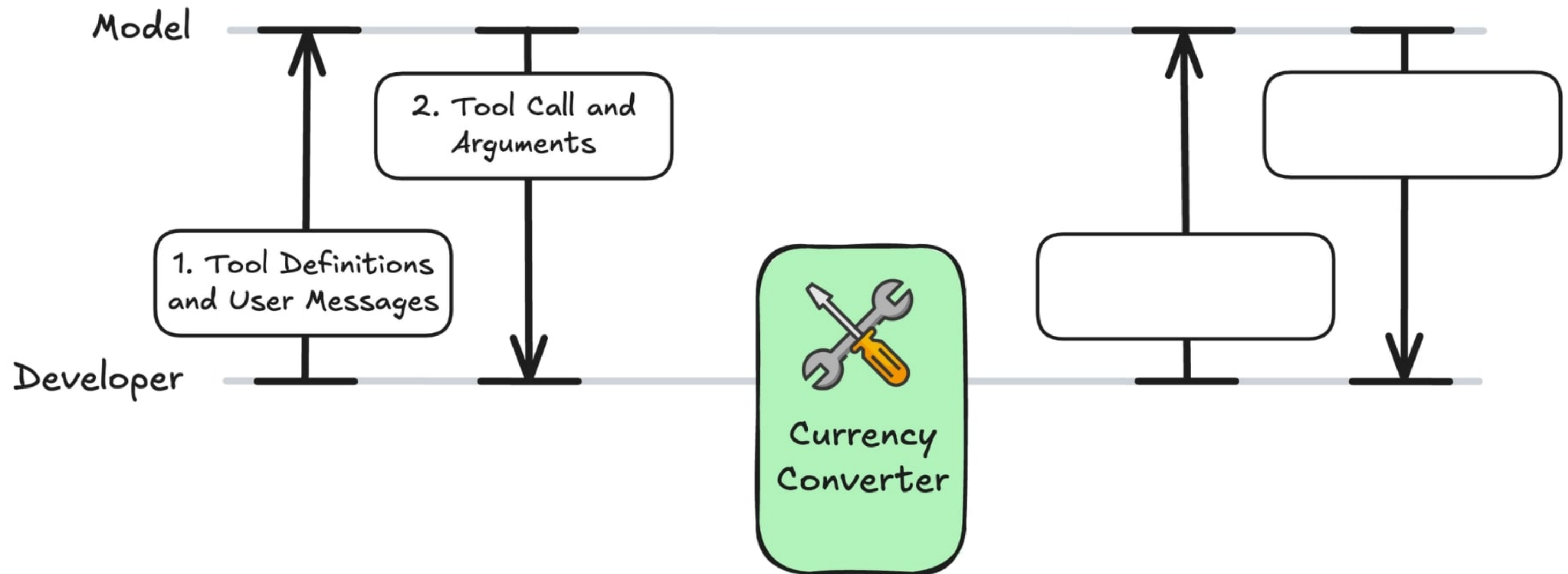
Function-Calling Tools



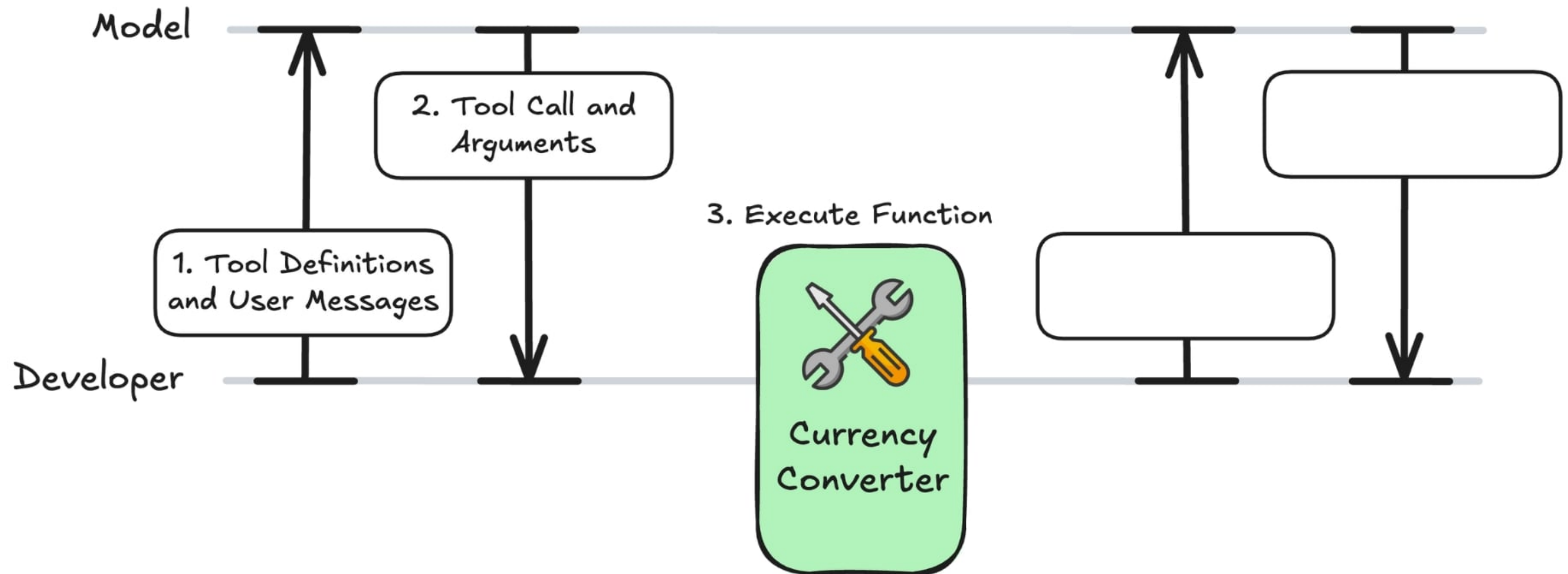
Function-Calling Tools



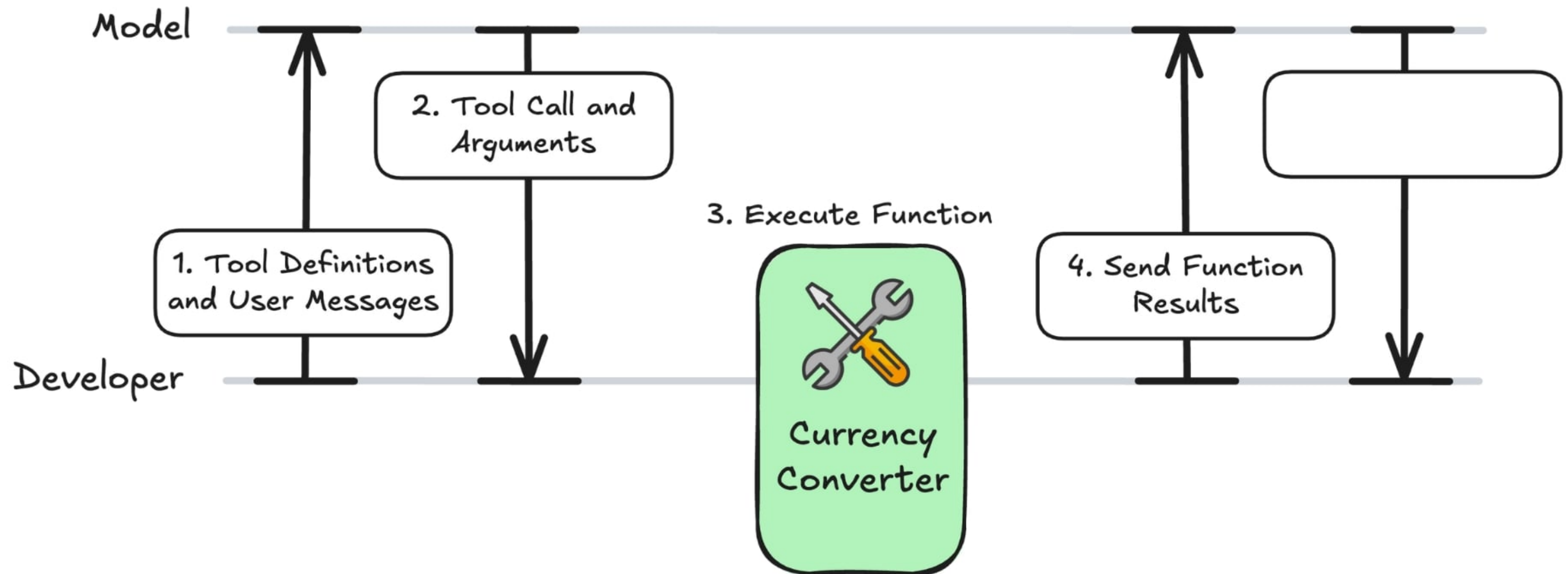
Function-Calling Tools



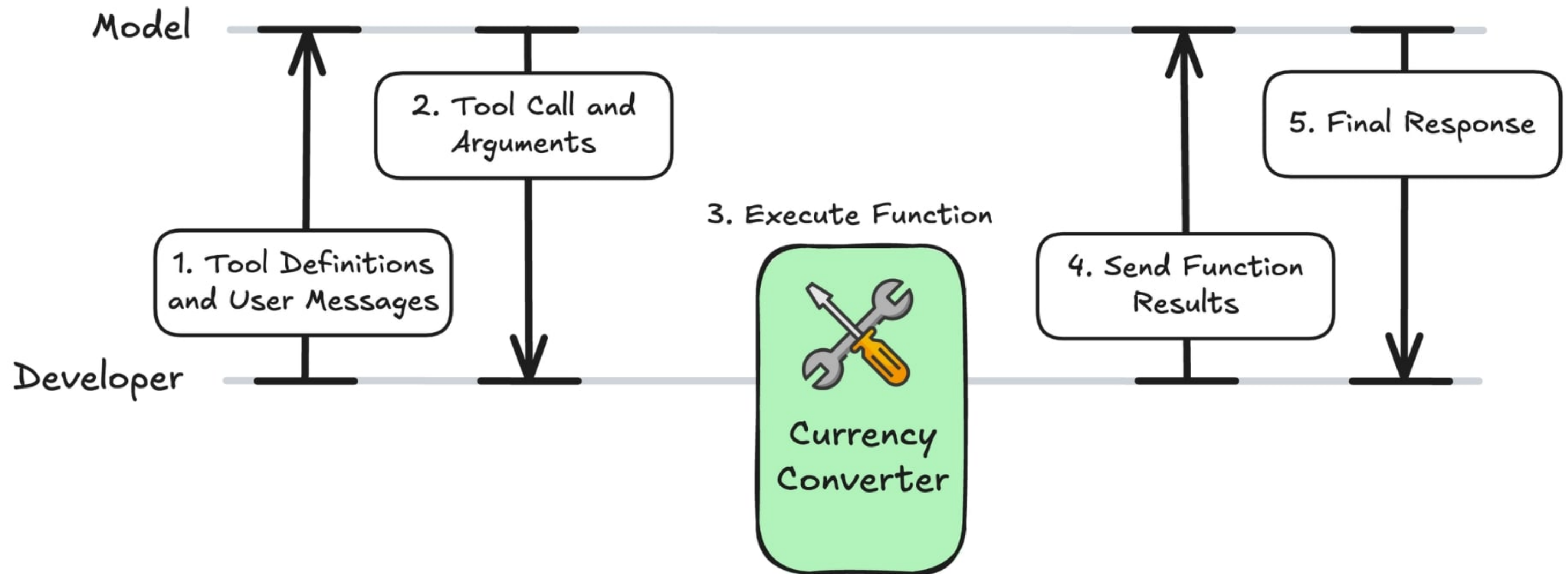
Function-Calling Tools



Function-Calling Tools



Function-Calling Tools



```
def convert_currency(amount, from_currency, to_currency):  
    url = f"https://api.frankfurter.dev/v1/latest?base={from_currency}&symbols={to_currency}"  
  
    try:  
        response = requests.get(url)  
        response.raise_for_status()  
  
        data = response.json()  
        rate = data['rates'].get(to_currency)  
  
        if rate is None:  
            return f"Could not find exchange rate for {from_currency} to {to_currency}"  
  
        converted_amount = amount * rate  
        return f"{amount} {from_currency} = {converted_amount:.2f} {to_currency} (Rate: {rate})"  
  
    except requests.exceptions.RequestException as e:  
        return f"Error converting currency: {str(e)}"
```

Our Function

```
print(convert_currency(100, "USD", "EUR"))
```

```
100 USD = 86.62 EUR (Rate: 0.86625)
```

Let's practice!

WORKING WITH THE OPENAI RESPONSES API

Function-Calling Tools: The Full Workflow

WORKING WITH THE OPENAI RESPONSES API



James Chapman

AI Curriculum Manager, DataCamp

```
def convert_currency(amount, from_currency, to_currency):
    url = f"https://api.frankfurter.dev/v1/latest?base={from_currency}&symbols={to_currency}"

    try:
        response = requests.get(url)
        response.raise_for_status()

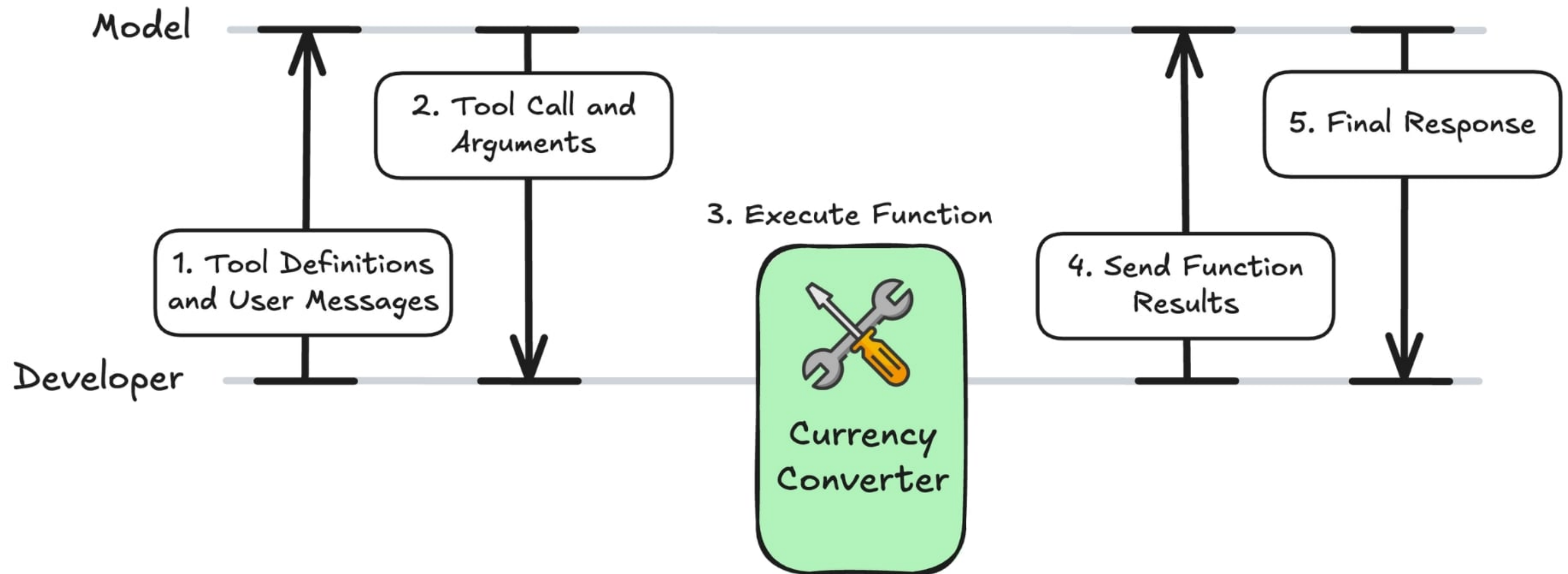
        data = response.json()
        rate = data['rates'].get(to_currency)

        if rate is None:
            return f"Could not find exchange rate for {from_currency} to {to_currency}"

        converted_amount = amount * rate
        return f"{amount} {from_currency} = {converted_amount:.2f} {to_currency} (Rate: {rate})"

    except requests.exceptions.RequestException as e:
        return f"Error converting currency: {str(e)}"
```

Function-Calling Tools



Tool Definitions: An LLM's Manual

```
tools = [  
  {  
    "type": "function",  
    "name": "convert_currency",  
    "description": "Convert an amount from one currency to another using real-time exchange rates.",  
    "parameters": {  
      "type": "object",  
      "properties": {  
        "amount": {"type": "number", "description": "The amount of money to convert"},  
        "from_currency": {"type": "string", "description": "The source currency code (e.g., 'USD', 'EUR')"},  
        "to_currency": {"type": "string", "description": "The target currency code (e.g., 'USD', 'EUR')"}  
      },  
      "required": ["amount", "from_currency", "to_currency"],  
      "additionalProperties": False  
    },  
  }  
]
```

```
response = client.responses.create(
    model="gpt-5.4-mini",
    input=[{"role": "user", "content": "How much is $100 in euros?"}],
    tools=tools
)

for item in response.output:
    print(item)
```

```
ResponseReasoningItem(id='rs_0a46aaf8050644c4006924978291a481909d5f50b8b9f7e959',
    summary=[], type='reasoning', content=None, encrypted_content=None, status=None)

ResponseFunctionToolCall(arguments='{"amount":100,"from_currency":"USD","to_currency":"EUR"}',
    call_id='call_xTbWJu06TiuFw84TK0hU9hRT', name='convert_currency',
    type='function_call',
    id='fc_0a46aaf8050644c40069249784fa248190874283efa2fc2c46', status='completed')
```

```
messages = [{"role": "user", "content": "How much is $100 in euros?"}]

response = client.responses.create(
    model="gpt-5.5",
    input=messages,
    tools=tools
)

messages += response.output
```

```
for item in response.output:
    if item.type == "function_call":
        if item.name == "convert_currency":
            currency_result = convert_currency(**json.loads(item.arguments))

            messages.append({
                "type": "function_call_output",
                "call_id": item.call_id,
                "output": json.dumps({"convert_currency": currency_result})
            })

response = client.responses.create(
    model="gpt-5.4-mini",
    instructions="Respond to the user input using the tool output.",
    tools=tools,
    input=messages,
)
```

```
print(response.output_text)
```

100 USD = 86.62 EUR (rate used: 1 USD = 0.86625 EUR).

Note: exchange rates fluctuate and actual amounts may differ due to fees or the provider you use. Want to convert a different amount or use a different currency?

Let's practice!

WORKING WITH THE OPENAI RESPONSES API